

RABBIT HOLES

Accelerate · Manipal University Jaipur

7

PHASE

Merging deeply

Strategies, why some merges make a commit, and teaching git to resolve a conflict once.



Contents

1	What a merge actually does	3
2	Fast-forward versus three-way	3
2.1	Fast-forward	3
2.2	Three-way merge	3
2.3	Controlling which one you get	4
3	Which button on GitHub	4
4	Merge strategies	5
4.1	Strategy options, which are different	5
5	Better conflict markers	6
6	rerere	6
7	Merge tools	7
8	Branching workflows	7
8.1	GitHub flow	7
8.2	Trunk-based development	8
8.3	Git flow	8
8.4	Merge queues	8
9	Reference	9

1 What a merge actually does

In the workshop you resolved a conflict by editing a file and running `git add`. That is the whole user-facing experience of merging, and it hides a fair amount of machinery worth understanding, because the machinery is what determines whether your project's history is readable in a year.

A merge combines two branches. Git does it in one of two ways, and which one you get depends entirely on the shape of the history.

2 Fast-forward versus three-way

2.1 Fast-forward

You branched off `main`, made two commits, and `main` has not moved since.

```
A---B  main
      \
      C---D  feature
```

To merge `feature` into `main`, git does not need to combine anything. It just moves the `main` label forward to `D`.

```
A---B---C---D  main, feature
```

That is a **fast-forward**. No merge commit is created, because nothing was merged. The history becomes a straight line, and afterwards there is no record that a branch ever existed.

2.2 Three-way merge

Now `main` has moved on while you worked.

```
A---B---E---F  main
      \
      C---D  feature
```

There is no way to move a label and get a correct answer. Git finds the **merge base**, the most recent commit both branches share, which is `B`. Then it compares three points: the base, and each of the two tips. Hence three-way.

Where only one side changed a region, git takes that change. Where both changed the same region differently, it stops and asks you. That is a conflict, and it is the only situation in which git ever asks.

The result is a **merge commit**, the only kind of commit with two parents:

```
A---B---E---F---M  main
      \           /
```

```
C---D---
```

WHY THIS EXISTS

Understanding that conflicts only happen when *both* sides changed the same region explains why the Phase 3 exercise worked. Everyone inserted a name at the same position in the credits block, so every branch after the first had both sides changing the same region. If people had added names in different places, git would have merged them silently.

It also explains the Phase 2 filename rule. Different files never share a region, so they can never conflict.

2.3 Controlling which one you get

```
git merge --no-ff feature
```

Forces a merge commit even when a fast-forward was possible. The branch stays visible in the history as a distinct piece of work.

```
git merge --ff-only feature
```

Refuses to merge unless it can fast-forward. Useful in scripts and as a safety check: it fails rather than silently creating a merge you did not intend.

```
git merge --squash feature
```

Takes all the changes, stages them, and makes no commit. You commit once yourself. The branch's individual commits are not recorded at all. This is what GitHub's "Squash and merge" button does.

3 Which button on GitHub

Every pull request offers three merge options, and teams argue about them. They map directly onto the above.

Option	What you get
Create a merge commit	Every commit from the branch, plus a merge commit. Full history, more noise.
Squash and merge	One commit on <code>main</code> containing everything. Clean history, individual steps lost.
Rebase and merge	Each commit replayed onto <code>main</code> , no merge commit. Linear history, hashes change.

NOTE

For a workshop, and for most student projects, **squash and merge** is the right default. Contributors are still learning to make tidy commits, and squashing means a messy branch still produces one clean entry on `main`.

For a project where contributors write careful, self-contained commits, squashing throws away real information, and a merge commit or rebase preserves it.

There is no universally correct answer, which is why the argument never ends. Pick one, write it in `CONTRIBUTING.md`, and stop discussing it.

4 Merge strategies

The strategy is the algorithm git uses to combine the trees.

Strategy	Use
<code>ort</code>	The default since Git 2.34. Handles renames and directory moves far better than what it replaced. You will almost never specify it. The old default. Still available, no longer necessary.
<code>recursive</code>	Older and simpler, handles only two heads.
<code>resolve</code>	Merges more than two branches at once. Refuses if there are conflicts. Rare.
<code>octopus</code>	Discards the other branch's changes entirely, keeps the merge commit as a record.
<code>ours</code>	For merging a project into a subdirectory of another.
<code>subtree</code>	

```
git merge -s ours old-branch
```

4.1 Strategy options, which are different

This is a genuine trap. `-s ours` is a strategy. `-X ours` is an option *to* a strategy, and it means something much narrower.

```
git merge -X ours feature
```

This does a normal merge, taking every non-conflicting change from both sides, and resolves only the *conflicting* regions in favour of your side.

TRAP

`-s ours` throws away everything from the other branch. `-X ours` keeps everything that did not conflict.

The letter is the only difference and the outcomes are wildly different. If you mean “take their version when we disagree” you want `-X theirs`. If you type `-s theirs` you will get an error, because it does not exist, which is the one piece of luck in this design.

5 Better conflict markers

The default conflict display shows you two versions. There is a better one.

```
git config --global merge.conflictStyle zdiff3
```

Now conflicts include the original, from before either side changed it:

```
<<<<<< HEAD
timeout = 30
|||||| base
timeout = 10
=====
timeout = 60
>>>>>> feature
```

The middle section changes everything. Without it you see two numbers and have to guess the intent. With it you can see that one side tripled the timeout and the other sextupled it, both were raising it from ten, and the answer is probably sixty rather than a random pick.

`diff3` is the older version of the same idea. `zdiff3` is tidier and available from Git 2.35 onwards. There is no reason not to turn it on.

6 rerere

Reuse recorded resolution. Git watches how you resolve a conflict and, if it ever sees the identical conflict again, resolves it the same way without asking.

```
git config --global rerere.enabled true
```

That is the entire setup. From then on it works quietly.

WHY THIS EXISTS

This sounds like a small convenience until you have a long-lived branch that you rebase onto a moving `main` every day. Without `rerere` you resolve the same three conflicts every single time, identically, forever. With it you resolve them once.

It also helps in the ordinary case of a merge that you abort, reconsider, and redo. Git remembers what you decided the first time.

Inspect what it has stored:

```
git rerere status
git rerere diff
git rerere forget <path>
```

`forget` is what you want when you resolved something wrongly and git is now cheerfully repeating your mistake.

7 Merge tools

For a conflict spanning many regions, editing markers by hand gets tiring.

```
git mergetool
```

This opens a three-pane view: their version, the merged result, yours. Configure which program it uses:

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd "code --wait $MERGED"
git config --global mergetool.keepBackup false
```

That last line stops git leaving `.orig` files everywhere, which is otherwise a persistent small annoyance.

NOTE

Editors have quietly made this obsolete for most people. VS Code, JetBrains IDEs and Neovim all detect conflict markers and offer “accept current, accept incoming, accept both” inline. That is usually faster than a separate three-pane tool.

Knowing what the markers mean still matters, because the buttons are just editing text and you need to know when “accept both” is wrong.

8 Branching workflows

The strategies above are mechanics. A workflow is the social agreement about which branches exist and what may be merged into them.

8.1 GitHub flow

One long-lived branch, `main`, which is always deployable. Every change is a short branch that becomes a pull request and is merged and deleted.

That is the whole workflow. It is what this repository uses, what most web projects use, and what you should default to.

8.2 Trunk-based development

Everyone commits to `main`, many times a day, with branches living hours rather than days. Unfinished work is hidden behind feature flags rather than in branches.

Requires strong automated testing, because there is no branch to hold broken work. Used heavily at large companies where long-lived branches cause more pain than feature flags do.

8.3 Git flow

Five kinds of branch: `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`. Designed in 2010 for software shipped as versioned downloads.

NOTE

Git flow is widely taught and, for most projects today, the wrong choice. Its author has since added a note to the original article saying it was designed for a world where you release versioned software on a schedule, and that continuously deployed web applications should use something simpler.

If you inherit it, fine. Do not adopt it for a student project. The overhead is real and the benefit only appears when you genuinely maintain several released versions at once.

8.4 Merge queues

On a busy repository, a pull request can be tested against a `main` that changes before it merges, so a green PR can still break `main`.

A merge queue solves this: pull requests enter a queue, each is tested against the result of everything ahead of it, and merges only if that passes. GitHub has this built in, under branch protection settings. It becomes worth the setup somewhere around the point where merge collisions happen weekly.

9 Reference

Goal	Command
Always record the branch	<code>git merge --no-ff <branch></code>
Refuse anything but a fast-forward	<code>git merge --ff-only <branch></code>
Combine into one commit yourself	<code>git merge --squash <branch></code>
Prefer my side on conflicts only	<code>git merge -X ours <branch></code>
Prefer their side on conflicts only	<code>git merge -X theirs <branch></code>
Discard their branch, keep the record	<code>git merge -s ours <branch></code>
See the original in conflicts	<code>git config merge.conflictStyle zdiff3</code>
Never resolve the same conflict twice	<code>git config rerere.enabled true</code>
Find where two branches diverged	<code>git merge-base main feature</code>
Back out of a merge	<code>git merge --abort</code>

CHECK YOURSELF

1. When does git fast-forward instead of making a merge commit?
2. What are the three inputs to a three-way merge?
3. Explain the difference between `-s ours` and `-X ours`.
4. What does `zdiff3` add to a conflict marker, and why is it useful?
5. Your branch conflicts with `main` the same way every day. What do you turn on?

Next: Phase 8, *Remotes and big repositories*, covering what a refspec is, how to work on several branches at once without stashing, and what to do when the repository is too large to clone comfortably.